

Teoria Współbieżności

Laboratorium 3

Problem ograniczonego bufora (problem producentów-konsumentów) oraz przetwarzanie potokowe z buforem

Kacper Bieniasz

21 października 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

2 Treść zadań

2.1 Zadanie 1

Dany jest bufor, do którego producent może wkładać dane, a konsument pobierać. Napisać program, który zorganizuje takie działanie producenta i konsumenta, w którym zapewniona będzie własność bezpieczeństwa i żywotności.

Zrealizować program:

1. przy pomocy metod `wait()`/`notify()`.
 - (a) dla przypadku 1 producent/1 konsument
 - (b) dla przypadku n_1 producentów/ n_2 konsumentów ($n_1 > n_2$, $n_1 = n_2$, $n_1 < n_2$)
 - (c) wprowadzić wywołanie metody `sleep()` i wykonać pomiary, obserwując zachowanie producentów/konsumentów
2. przy pomocy operacji `P()`/`V()` dla semafora:
 - (a) $n_1 = n_2 = 1$
 - (b) $n_1 > 1$, $n_2 > 1$

2.2 Zadanie 2

Zaimplementować rozwiązanie przetwarzania potokowego (Przykładowe założenia: bufor rozmiaru 100, 1 producent, 1 konsument, 5 uszeregowanych procesów przetwarzających.) Od czego zależy prędkość obróbki w tym systemie? Rozwiązanie za pomocą semaforów lub monitorów (dowolnie). W implementacji nie jest dozwolone korzystanie/implementowanie własnych kolejek FIFO, należy używać tylko mechanizmu monitorów lub semaforów!

3 Realizacja zadania nr 1

Kod w języku Java (1) przedstawia wersję programu realizującą wykorzystanie metod `wait()/notify()`. Program implementuje różną liczbę producentów i konsumentów. Dodatkowo trzeba było dodać zmienną liczbę iteracji (pobrań z bufora, konsumpcji oraz dokładania do bufora, produkcji) tak, aby zachodziła relacja $n1 \cdot \text{liczba produkcji} = n2 \cdot \text{liczba konsumpcji}$. Do nadzorowania pracy wątków korzystam z klasy **ExecutorService**. Klasy **Producer** i **Consumer** są do siebie bardzo podobne. Każdy obiekt klasy otrzymuje swój identyfikator, aby można było stwierdzić, który producent/konsument dokłada lub zabiera z bufora. Obie klasy wykonują metodę `put()/get()` na buforze odpowiednią ilość razy. Po każdym wykonaniu czekają pewną ilość czasu. W celu mniej przewidywalnego zachowania można losować ilość czasu pomiędzy kolejnymi iteracjami. Klasa **Buffer** realizuje mechanizm oczekiwania na warunek dla dwóch przypadków, jednego kiedy producent nie może dołożyć (bufor jest pełny), drugiego kiedy konsument nie może zabrać z bufora (bufor jest pusty). W obu przypadkach producent lub konsument muszą czekać.

W przypadku $n1 = n2 = 1$ zachowanie programu przebiega jako cykl:

- Producent 0 kładzie: n
- Konsument 0 pobrał: n
- Konsument 0 czeka...

Problem czekania konsumenta znika po kilku pierwszych operacjach jednak można to zmienić czekając losową liczbę sekund w metodzie `sleep()`.

W przypadku gdy $n1 > n2$, obserwujemy szybkie zapełnienie się bufora, przez co producenci muszą czekać, natomiast konsumenci cały czas mogą dokonywać pobrań. Kiedy $n1 = n2$ i $n1 \neq 1$ zachowanie jest podobne jak w przypadku $n1 = n2 = 1$, każdemu konsumentowi odpowiada jeden producent więc wszystko przebiega jednostajnie. Przypadek kiedy $n1 < n2$ wygląda podobnie do $n1 > n2$, tylko w tym przypadku to konsument musi czekać, ponieważ bufor jest pusty.

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3 class Producer extends Thread {
4     private Buffer _buf;
5     private int iter;
6     private int id;
7     public Producer(int id, Buffer buffer, int iter) {
8         this._buf = buffer;
9         this.iter = iter;
10        this.id = id;
11    }
12    public void run() {
13        for (int i = 0; i < iter; i++) {
14            _buf.put(id, i);
15            try {
16                sleep(50);
17            } catch (InterruptedException e) {
18                Thread.currentThread().interrupt();
19            }
20        }
21    }
22 }
23 class Consumer extends Thread {
24     private Buffer _buf;
25     private int iter;
26     private int id;
27     public Consumer(int id, Buffer buffer, int iter) {
28         this._buf = buffer;
29         this.iter = iter;
```

```

30     this.id = id;
31 }
32 public void run() {
33     for (int i = 0; i < iter; i++) {
34         System.out.println("Konsument " + id + " pobrał: " + _buf.get(id));
35         try {
36             sleep(50);
37         } catch (InterruptedException e) {
38             Thread.currentThread().interrupt();
39         }
40     }
41 }
42 }
43 class Buffer {
44     private final int[] buffer;
45     private int count = 0, in = 0, out = 0;
46     public Buffer(int size) {
47         buffer = new int[size];
48     }
49     public synchronized void put(int id, int value) {
50         while (count == buffer.length) {
51             try {
52                 System.out.println("Producent " + id + " czeka...");
53                 wait();
54             } catch (InterruptedException e) {
55                 Thread.currentThread().interrupt();
56             }
57         }
58         System.out.println("Producer " + id + " kładzie: " + value);
59         buffer[in] = value;
60         in = (in + 1) % buffer.length;
61         count++;
62         notify();
63     }
64     public synchronized int get(int id) {
65         while (count == 0) {
66             try {
67                 System.out.println("Konsument " + id + " czeka...");
68                 wait();
69             } catch (InterruptedException e) {
70                 Thread.currentThread().interrupt();
71             }
72         }
73         int value = buffer[out];
74         out = (out + 1) % buffer.length;
75         count--;
76         notify();
77         return value;
78     }
79 }
80 public class PKmon {
81     public static void main(String[] args) {
82         Buffer buffer = new Buffer(100);
83         int n1 = 1;
84         int n2 = 4;
85         int iter1 = 400;
86         int iter2 = 100;
87         ExecutorService executorService = Executors.newFixedThreadPool(n1 + n2);
88         for (int i = 0; i < n1; i++) {
89             executorService.submit(new Producer(i, buffer, iter1));
90         }
91         for (int i = 0; i < n2; i++) {

```

```

92         executorService.submit(new Consumer(i,buffer, iter2));
93     }
94     executorService.shutdown();
95 }
96 }

```

Kod Java 1: Wersja wykorzystująca metody wait()/notify()

Zastosowanie semaforów przedstawia kod (2). Wykorzystanie semaforów oraz operacji P() i V() na nich skutkuje takim samym zachowaniem programu jak w przypadku użycia metod wait() oraz notify(). W celu użycia semaforów dodajemy klasę **Semaphore**, które implementuje standardową wersję semafora. Najwięcej modyfikacji otrzymała klasa **Buffer**, musi ona przechowywać, aż trzy semafony. Jeden jako mutex i dwa odpowiadające za pełny i pusty bufor. W związku z tym zmianą uległy jeszcze metody put() oraz get() wykorzystujące aktualnie semafony.

```

1  import java.util.concurrent.ExecutorService;
2  import java.util.concurrent.Executors;
3  class Producer extends Thread {
4      private Buffer _buf;
5      private int iter;
6      private int id;
7      public Producer(int id, Buffer buffer, int iter) {
8          this._buf = buffer;
9          this.iter = iter;
10         this.id = id;
11     }
12     public void run() {
13         for (int i = 0; i < iter; i++) {
14             _buf.put(id, i);
15             try {
16                 sleep(50);
17             } catch (InterruptedException e) {
18                 Thread.currentThread().interrupt();
19             }
20         }
21     }
22 }
23 class Consumer extends Thread {
24     private Buffer _buf;
25     private int iter;
26     private int id;
27     public Consumer(int id, Buffer buffer, int iter) {
28         this._buf = buffer;
29         this.iter = iter;
30         this.id = id;
31     }
32     public void run() {
33         for (int i = 0; i < iter; i++) {
34             System.out.println("Konsument " + id + " pobrał: " + _buf.get(id));
35             try {
36                 sleep(50);
37             } catch (InterruptedException e) {
38                 Thread.currentThread().interrupt();
39             }
40         }
41     }
42 }
43 class Buffer {
44     private final int[] buffer;
45     private int in = 0, out = 0;
46     private Semaphore mutex = new Semaphore(1);
47     private Semaphore empty;

```

```

48     private Semaphore full = new Semaphore(0);
49     public Buffer(int size) {
50         buffer = new int[size];
51         empty = new Semaphore(size);
52     }
53     public void put(int id, int value) {
54         empty.P();
55         mutex.P();
56         System.out.println("Producer " + id + " kładzie: " + value);
57         buffer[in] = value;
58         in = (in + 1) % buffer.length;
59         mutex.V();
60         full.V();
61     }
62     public int get(int id) {
63         full.P();
64         mutex.P();
65         int value = buffer[out];
66         out = (out + 1) % buffer.length;
67         mutex.V();
68         empty.V();
69         return value;
70     }
71 }
72 class Semaphore {
73     private int value;
74     public Semaphore(int value) {
75         this.value = value;
76     }
77     public synchronized void P() {
78         while (value <= 0) {
79             try {
80                 wait();
81             } catch (InterruptedException e) {
82                 Thread.currentThread().interrupt();
83             }
84         }
85         value--;
86     }
87     public synchronized void V() {
88         value++;
89         notify();
90     }
91 }
92 public class PKmon {
93     public static void main(String[] args) {
94         Buffer buffer = new Buffer(100);
95         int n1 = 3;
96         int n2 = 1;
97         int iter1 = 100;
98         int iter2 = 300;
99         ExecutorService executorService = Executors.newFixedThreadPool(n1 + n2);
100         for (int i = 0; i < n1; i++) {
101             executorService.submit(new Producer(i, buffer, iter1));
102         }
103         for (int i = 0; i < n2; i++) {
104             executorService.submit(new Consumer(i,buffer, iter2));
105         }
106         executorService.shutdown();
107     }
108 }

```

4 Realizacja zadania nr 2

Kod (3) zawiera implementację przetwarzania potokowego z buforem. Wprowadziłem randomizację czasu wytworzenia i przetworzenia. W tym programie korzystam z wbudowanego mechanizmu semaforów. Program zawiera klasę **Producer**, który wytwarza i zwalnia semafor dla pierwszego procesu w potoku. Klasa **Processor**, która przetwarza dane zajmując i zwalniając odpowiednie semafony. Klasa **Consumer** zajmuje semafor jeśli, który będzie wolny po przetworzeniu przez wszystkie procesy przejściowe. Wprowadzenie randomizacji czasu spowodowało ciekawe zachowanie procesów, np pierwsze trzy kończą działanie dość szybko, a pozostałe dwa działają naprzemiennie. Prędkość przetwarzania danych w systemie zależy od najwolniejszego procesu w potoku. Jest to właśnie związane z relacją Lamporta ponieważ, pewne procesy muszą się wykonać przed innymi.

```

1 import java.util.concurrent.Semaphore;
2 public class PipelineProcessing {
3     private static final int BUFFER_SIZE = 100;
4     private static final int NUM_PROCESSORS = 5;
5     private static final int[] buffer = new int[BUFFER_SIZE];
6     private static final Semaphore[] semaphores = new Semaphore[BUFFER_SIZE + 1];
7     public static void main(String[] args) {
8         for (int i = 0; i < semaphores.length; i++) {
9             semaphores[i] = new Semaphore(0);
10        }
11        Thread producerThread = new Thread(new Producer());
12        Thread consumerThread = new Thread(new Consumer());
13        producerThread.start();
14        for (int i = 0; i < NUM_PROCESSORS; i++) {
15            new Thread(new Processor(i, i, i + 1)).start();
16        }
17        consumerThread.start();
18    }
19    static class Producer implements Runnable {
20        @Override
21        public void run() {
22            try {
23                for (int i = 0; i < BUFFER_SIZE; i++) {
24                    buffer[i] = i;
25                    System.out.println("Producent wytworzył " + i + " na pozycji " + i);
26                    semaphores[0].release();
27                    Thread.sleep((int)(Math.random()*50));
28                }
29            } catch (InterruptedException e) {
30                Thread.currentThread().interrupt();
31            }
32        }
33    }
34    static class Processor implements Runnable {
35        private final int id;
36        private final int inIndex;
37        private final int outIndex;
38        Processor(int id, int inIndex, int outIndex) {
39            this.id = id;
40            this.inIndex = inIndex;
41            this.outIndex = outIndex;
42        }
43        @Override
44        public void run() {

```

```

45         try {
46             for (int i = 0; i < BUFFER_SIZE; i++) {
47                 semaphores[inIndex].acquire();
48                 if (buffer[i] == -1) break;
49                 System.out.println("Procesor " + id + ": przetwarzanie danych " + buffer[i]);
50                 buffer[i] += 1;
51                 semaphores[outIndex].release();
52                 Thread.sleep((int)(Math.random()*50));
53             }
54         } catch (InterruptedException e) {
55             Thread.currentThread().interrupt();
56         }
57     }
58 }
59 static class Consumer implements Runnable {
60     @Override
61     public void run() {
62         try {
63             for (int i = 0; i < BUFFER_SIZE; i++) {
64                 semaphores[NUM_PROCESSORS].acquire();
65                 int data = buffer[i];
66                 System.out.println("Konsument: odebrano dane " + data + " na pozycji " + i);
67                 Thread.sleep((int)(Math.random()*50));
68             }
69         } catch (InterruptedException e) {
70             Thread.currentThread().interrupt();
71         }
72     }
73 }
74 }

```

Kod Java 3: Przetwarzanie potokowe z buforem

5 Wnioski

Rozwiązanie problemu ograniczonego bufora i przetwarzania potokowego przy użyciu mechanizmów wait()/notify() oraz semaforów P()/V() skutecznie zapewnia bezpieczeństwo i żywotność systemu. Synchronizacja przy użyciu tych metod pozwala na bezpieczny dostęp do bufora i zapobiega sytuacjom, w których producenci próbują dodać dane do pełnego bufora, a konsumenci próbują pobrać dane z pustego bufora. Dzięki temu system działa stabilnie, nawet przy różnych proporcjach liczby producentów i konsumentów. Wydajność systemu zależy od relacji między liczbą producentów i konsumentów oraz od rozmiaru bufora. Gdy liczba producentów jest większa od liczby konsumentów, bufor szybko się zapełnia, co powoduje oczekiwanie producentów na wolne miejsce. Odwrotnie, gdy konsumentów jest więcej niż producentów, zdarza się, że konsumenci muszą czekać na dostępność nowych danych. Zastosowanie odpowiednio dużego bufora zmniejsza ryzyko blokowania procesów i poprawia płynność przetwarzania. Dodatkowe wprowadzenie losowych opóźnień (sleep()) pozwala lepiej symulować rzeczywiste warunki pracy systemu, pokazując, jak zmienne prędkości przetwarzania mogą wpływać na przepływ danych. Właściwe zarządzanie czasem przetwarzania i synchronizacja pozwalają uniknąć zakleszczeń oraz zapewnić efektywność działania systemu.

Literatura

- [1] Per Brinch Hansen: Java Insecure Parallelism. ACM SIGPLAN Notices, April 1999, s. 38-45.
- [2] Thomas W. Christopher, George K. Thiruvathukal: High-Performance Java Platform Computing, Prentice Hall, Luty 2001.